Unit 2

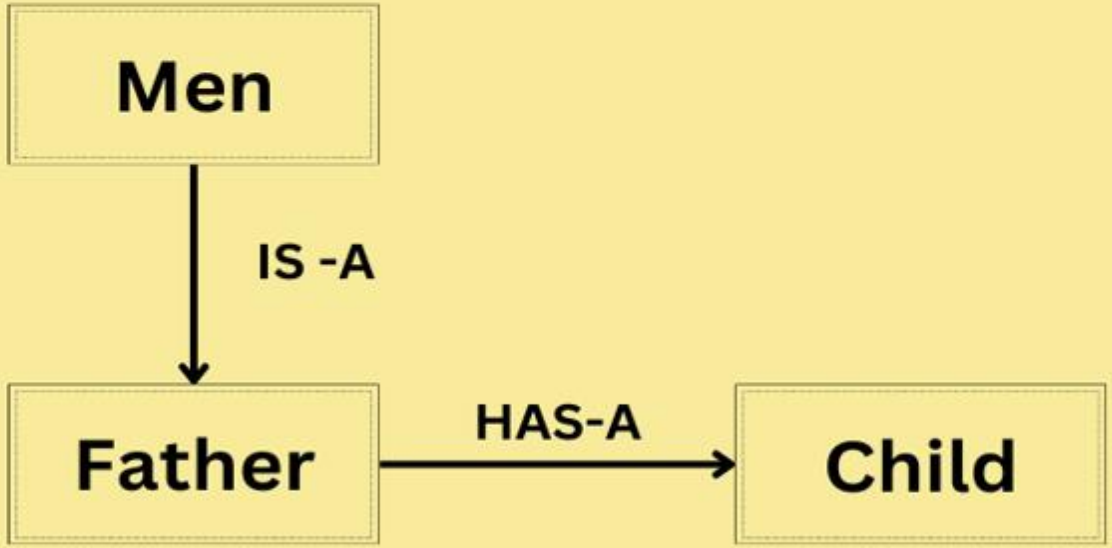
INHERITANCE AND INTERFACES

Inheritance – Super classes- sub classes –Protected members – constructors in sub classes- the Object class – abstract classes and methods- final methods and classes – Interfaces – defining an interface, implementing interface, differences between classes and interfaces and extending interfaces - Object cloning -inner classes, Array Lists - Strings

Inheritance - Introduction

- Java is an object-oriented programming language that supports inheritance.
- Inheritance is the ability to inherit the features and functionalities from another class.
- It supports the reuse of the same code.
- It allows the programmer to define a class in terms of another class. A relationship in Java means different relations between two or more classes.
- In Java, we can have a has-a relationship between classes, otherwise known as composition.
- A composition is a form of association, where an association is a relation between two classes that is established using their objects.

Has-A Relationship in Java



Java

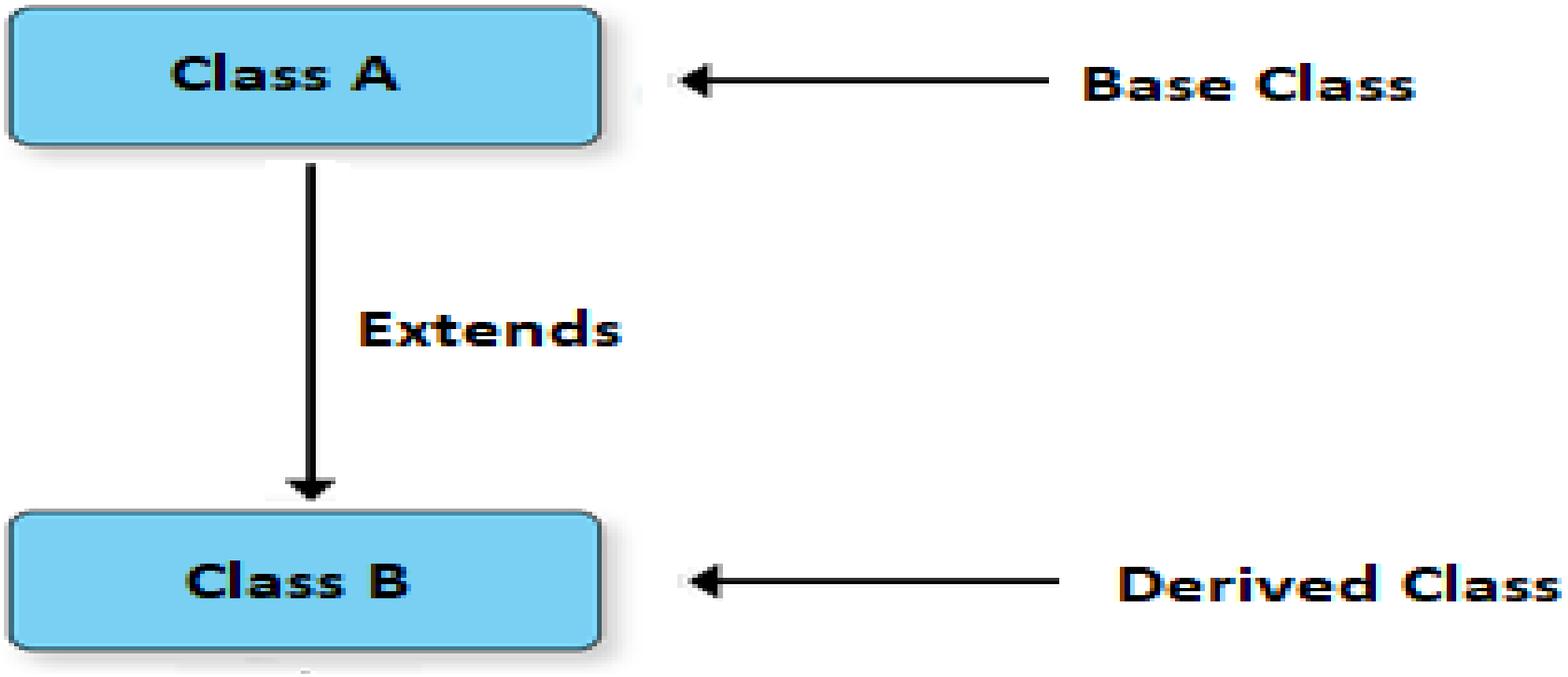
Inheritance

- Inheritance is one of the cornerstones of object-oriented programming because it allows the creation of hierarchical classifications.
- Using inheritance, you can create a general class that defines traits common to a set of related items.
- In the terminology of Java, a class that is inherited is called a superclass
- The class that does the inheriting is called a subclass.
- To inherit a class, you simply incorporate the definition of one class into another by using the ***extends*** keyword

Super classes and Sub classes

- A class that is derived from another class is called a subclass (also a derived class, extended class, or child class).
- The class from which the subclass is derived is called a superclass (also a base class or a parent class)

Single Inheritance



Single Inheritance

- Single Inheritance is also known as basic inheritance and it is one of the types of inheritance in object oriented programming.
- In single inheritance, the child or derived class inherits the properties and behaviours directly from the single parent class.
- In this inheritance, the subclass has access to all the members from the parent class except the private members.

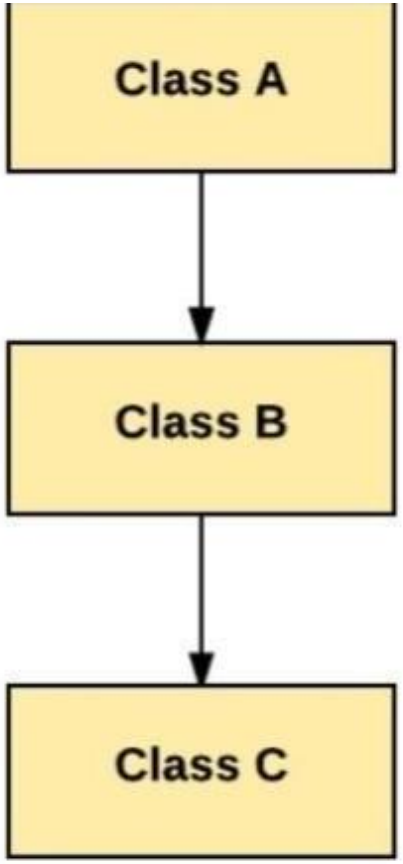
Single Inheritance

- Single level inheritance is used in java to improve code organisation and maintenance .
- It also helps in code reuse as the subclass can inherit and reuse the code from the parent class and can override, add its own features too.
- This approach reduces redundancy in the code as we are not writing code again and again

Multi-level Inheritance

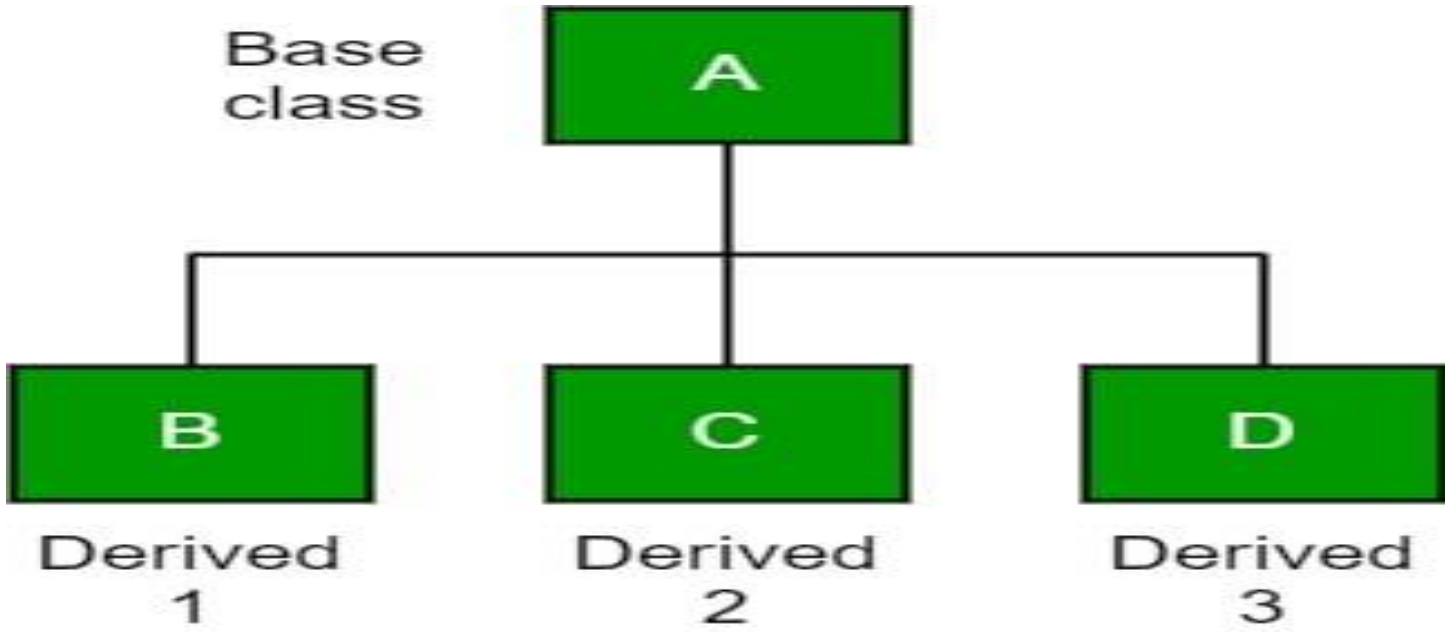
- In Multi-Level Inheritance in Java, a class extends to another class that is already extended from another class.
- For example, if there is a class A that extends class B and class B extends from another class C, then this scenario is known to follow Multi-level Inheritance.

Multilevel Inheritance



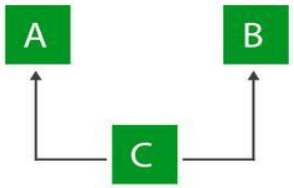
Hierarchical Inheritance

- In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.



Multiple Inheritance

- In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes.
- Java does not support multiple inheritances with classes.
- The Diamond Problem is a common issue that arises with multiple inheritance. When a class inherits from multiple parent classes that have the same method or field, the compiler gets confused on which one to choose.

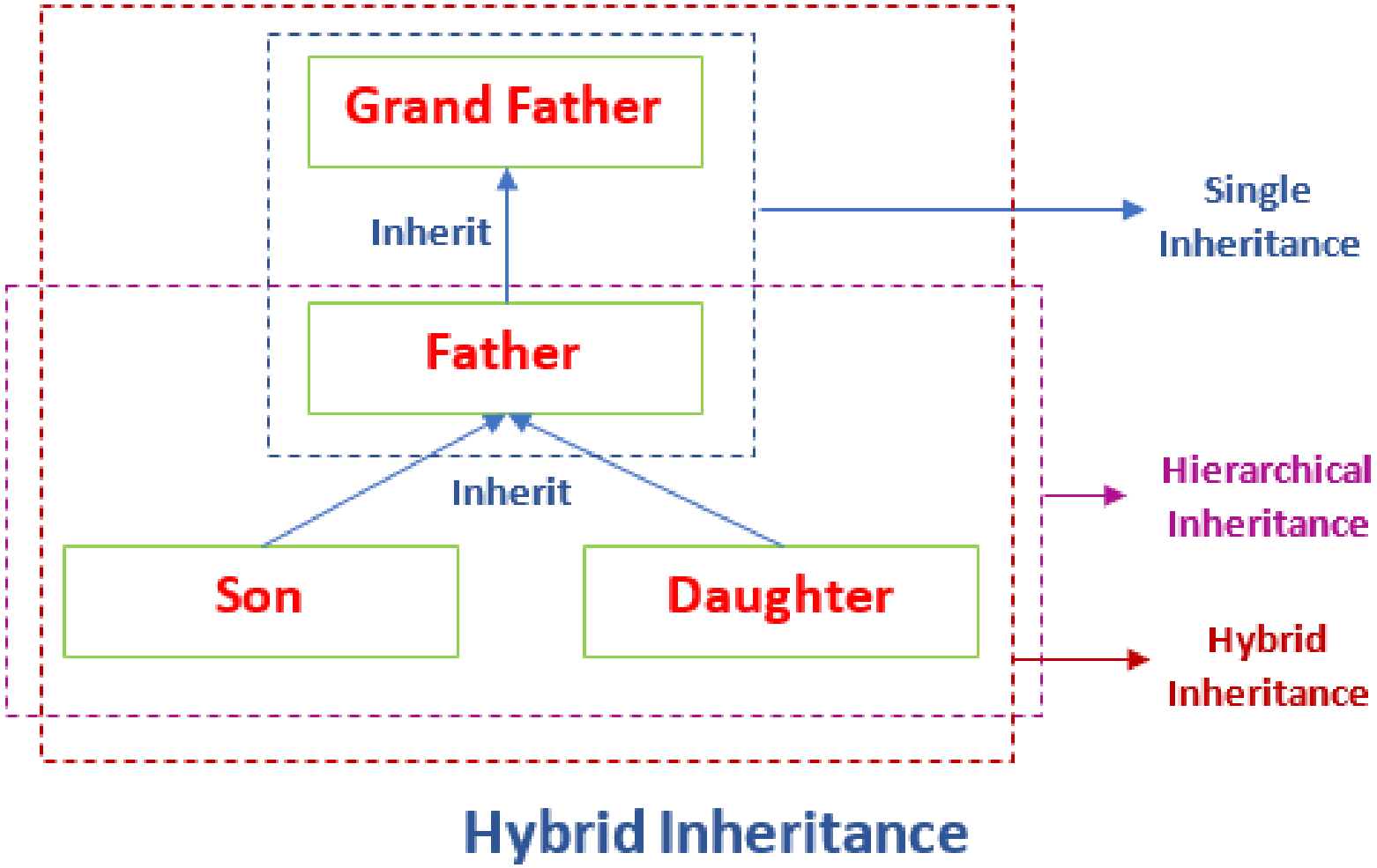


Multiple Inheritance

Hybrid Inheritance

- In general, the meaning of hybrid (mixture) is made of more than one thing.
- In Java, the hybrid inheritance is the composition of two or more types of inheritance.
- The main purpose of using hybrid inheritance is to modularize the code into well-defined classes.
- It also provides the code reusability.

Hybrid Inheritance Example



Interface in Java

- Using interface, you can specify what a class must do, but not how it does it.
- Once it is defined, any number of classes can implement an interface.
- Also, one class can implement any number of interfaces.
- By providing the interface keyword, Java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.

Interfaces (Cont..)

An interface is defined much like a class. This is a simplified general form of an interface:

```
access interface name {
return-type method-name1(parameter-list);
return-type method-name2(parameter-list);
type varname1 = value;
type varname2 = value;
return-type method-nameN(parameter-list);
type varnameN = value;
}
```

Implementing Interfaces

- Once an interface has been defined, one or more classes can implement that interface.
- To implement an interface, include the implements clause in a class definition, and then create the methods defined by the interface.

```
class classname [extends superclass] [implements interface
[,interface...]] {
// class-body
}
```

Partial Implementations

If a class includes an interface but does not fully implement the methods defined by that

interface, then that class must be declared as abstract. For example:

```
abstract class Incomplete implements Callback {
int a, b;
void show() {
System.out.println(a + " " + b);
}
//...
}
```

Abstract Methods in Java

- Any class that contains one or more abstract methods must also be declared abstract.
- If a class contains an abstract method it needs to be abstract and vice versa is not true.
- If a non-abstract class extends an abstract class, then the class must implement all the abstract methods of the abstract class else the concrete class has to be declared as abstract as well.

Method Overloading in Java

- When a class has two or more methods by the same name but different parameters, at the time of calling based on the parameters passed respective method is called as method overloading.
- If we have to perform only one operation, having same name of the methods increases the readability of the program.
- There are two ways to overload the method in java
  - By changing number of arguments
  - By changing the data type

Method Overriding in Java

- In method overriding, Super class and subclass have methods with the same name including parameters.
- JVM calls the respective method based on the object used to call the method.
- If an object of a parent class is used to invoke the method, then the version in the parent class will be executed, but if an object of the subclass is used to invoke the method, then the version in the child class will be executed.
- In other words, it is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.
- Method overriding is one of the ways by which Java achieves Run Time Polymorphism.

Overloading	Overriding
Must have at least two methods by same name in same class.	Must have at least one methods by same name in both Parent and child class.
Must have different number of parameters.	Must have same number of parameters.
If number of parameters are same then must have different types of parameters.	Types of parameters also must be same.
Overloading known as compile time polymorphism.	Overriding knows as run time polymorphism.

Extending Interfaces

- An interface can extend other interfaces, just as a class subclass or extend another class.
- A class can extend only one other class, an interface can extend any number of interfaces.
- The interface declaration includes a comma-separated list of all the interfaces that it extends.

Protected members

- The **protected** keyword in Java refers to one of its access modifiers. The methods or data members declared as protected can be accessed from
- Within the same class.
- Subclasses of the same packages.
- Different classes of the same packages.
- Subclasses of different packages.

Protected Members (Cont ..)

- If one wishes to access a protected modifier outside a package, then inheritance is to be applied.
- Protecting a constructor prevents the users from creating the instance of the class, outside the package.
- During overriding, when a variable or method is protected, it can be overridden to other subclass using either a public or protected modifier only.
- Outer class and interface cannot be protected.

Constructors in sub classes

- super ()keyword is used to access the parent class constructor
- super () can call both parametric as well as non-parametric constructors depending upon the situation.
- Call to super() must be first statement in Derived Class constructor.
- If a constructor does not explicitly invoke a superclass constructor by using super(), the Java compiler automatically inserts a call to the no-argument constructor of the superclass.

super keyword

- The super keyword in Java is a reference variable that is used to refer parent class objects.
- super can be used to call parent class’ variables and methods

Object Class

- The **Object class** is the parent class of all the classes in java by default. In other words, it is the topmost class of java.
- The Object class is beneficial if you want to refer any object whose type you don't know.
- The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc.

Some Methods in Object class

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.

Object Cloning in Java

- The **object cloning** is a way to create exact copy of an object.
- The **clone()** method of Object class is used to clone an object.
- The **java.lang.Cloneable** interface must be implemented by the class whose object clone we want to create. If we don't implement Cloneable interface, clone() method generates **CloneNotSupportedException**.

Object Cloning in Java (Cont..)

- It is the easiest and most efficient way for copying objects, especially if we are applying it to an already developed or an old project.
- Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.
- The **clone() method** is defined in the Object class.

final methods and classes

- **Final methods:** When a method is declared as final, it cannot be overridden by a subclass.
- **Final classes:** When a class is declared as final, it cannot be extended by a subclass.
- **Final variable:** To create constant variable

Uses of final in Java

- **Ensures Data Immutability:** Once a reference or variable is marked using the final keyword, its value cannot be further modified after it gets assigned, ensuring the stored data's unchangeability.
- **Improves Performance and Productivity:** Another use of final class in Java is improving the performance by optimizing the codes more efficiently.
- **Simplification of the Codes:** Developers make the codes easily understandable and quickly reasonable by declaring the methods, variables, or classes as final, simplifying the code debugging and analysis procedures.
- **Enhances Security Aspects:** It prevents malicious activities and modifications to the stored data.
- **Promotes Reusability of Codes:** Subclasses cannot override the final class values, allowing the reuse of codes and reducing duplicate implementation

Abstract Method

- A method declared using the abstract keyword within an abstract class and does not have a definition (implementation) is called an abstract method.
- When we need just the method declaration in a super class, it can be achieved by declaring the methods as abstracts.
- Abstract method is also called subclass responsibility as it doesn't have the implementation in the super class. Therefore a subclass must override it to provide the method definition.

Abstract Method (Cont..)

- An abstract method do not have a body (implementation), they just have a method signature (declaration). The class which extends the abstract class implements the abstract methods.
- If a non-abstract (concrete) class extends an abstract class, then the class must implement all the abstract methods of that abstract class. If not the concrete class has to be declared as abstract as well.

Syntax for abstract methods

abstract return_type method_name([argument-list]);

Inner Classes

- Java inner class or nested class is a class that is declared inside the class or interface.
- We use inner classes to logically group classes and interfaces in one place to be more readable and maintainable.
- Additionally, it can access all the members of the outer class, including private data members and methods.

Syntax

```
class Java_Outer_class{
 //code
 class Java_Inner_class{
  //code
 }
}
```

Advantages of Inner Class (Nested Class)

- Nested classes represent a particular type of relationship that is it can access all the members (data members and methods) of the outer class, including private.
- Nested classes are used to develop more readable and maintainable code because it logically group classes and interfaces in one place only.

Static Inner Class

- A static nested class is one that has the **static** modifier applied.
- Because it is static, it must access the **non-static members of its enclosing class through an object**. That is, it cannot refer to non-static members of its enclosing class directly.
- Because of this restriction, **static nested classes are seldom used**.
- The most important type of nested class is the *inner* class. An inner class is a non-static nested class.
- It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.

Private Inner Class

- Unlike a "regular" class, an inner class can be private.
- If you don't want outside objects to access the inner class, declare the class as private

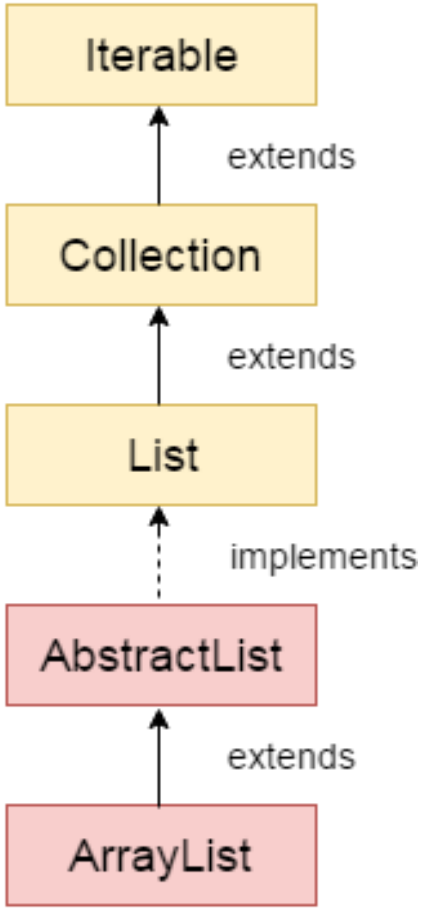
Array Lists

- Java ArrayList class uses a dynamic array for storing the elements.
- It is like an array, but there is no size limit. We can add or remove elements anytime. So, it is much more flexible than the traditional array.
- It is found in the java.util package.
- The ArrayList in Java can have the duplicate elements also.
- It implements the List interface so we can use all the methods of the List interface here.

ArrayLists (Cont..)

- As shown in the diagram, the Java ArrayList class extends AbstractList class which implements the List interface.

- The List interface extends the Collection and Iterable interfaces in hierarchical order.



Constructors of ArrayList

Constructor	Description
ArrayList()	It is used to build an empty array list.
ArrayList(Collection<? extends E> c)	It is used to build an array list that is initialized with the elements of the collection c.
ArrayList(int capacity)	It is used to build an array list that has the specified initial capacity.

Methods of ArrayList

void add(int index, E element)	It is used to insert the specified element at the specified position in a list.
boolean add(E e)	It is used to append the specified element at the end of a list.
boolean addAll(Collection<? extends E> c)	It is used to append all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
boolean addAll(int index, Collection<? extends E> c)	It is used to append all the elements in the specified collection, starting at the specified position of the list.
void clear()	It is used to remove all of the elements from this list.
void ensureCapacity(int requiredCapacity)	It is used to enhance the capacity of an ArrayList instance.

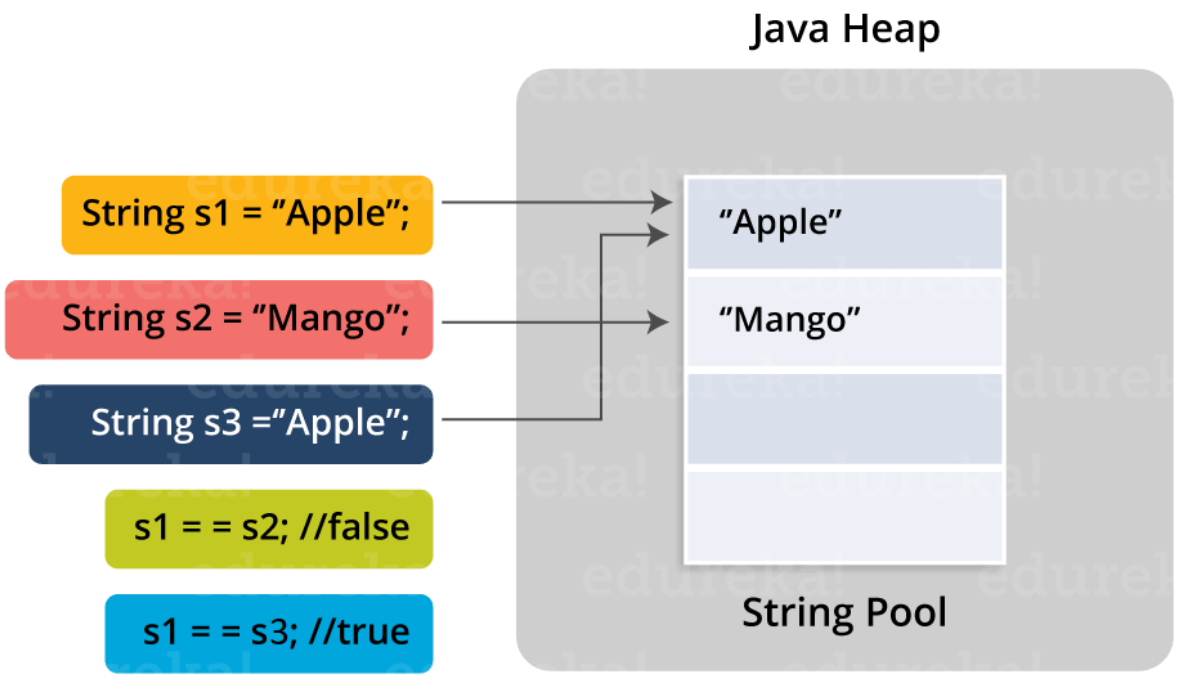
Strings

- In Java, a string is an object that represents a sequence of characters or char values.
- The *java.lang.String* class is used to create a Java string object.
- There are two ways to create a String object:

1. **By string literal** : Example: String s=“Hello”;
2. **By new keyword** : String s=new String(“Hello”);

Java String Pool

- Java String pool refers to collection of Strings which are stored in heap memory.



Built in String Functions in Java

Method	Description	Return Type
charAt()	Returns the character at the specified index (position)	char
codePointAt()	Returns the Unicode of the character at the specified index	int
codePointBefore()	Returns the Unicode of the character before the specified index	int
codePointCount()	Returns the number of Unicode values found in a string.	int
compareTo()	Compares two strings lexicographically	int
compareToIgnoreCase()	Compares two strings lexicographically, ignoring case differences	int
concat()	Appends a string to the end of another string	String
contains()	Checks whether a string contains a sequence of characters	boolean
contentEquals()	Checks whether a string contains the exact same sequence of characters of the specified CharSequence or StringBuffer	boolean
copyValueOf()	Returns a String that represents the characters of the character array	String
endsWith()	Checks whether a string ends with the specified character(s)	boolean
equals()	Compares two strings. Returns true if the strings are equal, and false if not	boolean
equalsIgnoreCase()	Compares two strings, ignoring case considerations	boolean
getBytes()	Encodes this String into a sequence of bytes using the named charset, storing the result into a new byte array	byte[]